Mobile App development practical

Overview

This is a filter interview, and only if you demonstrate that you possess good, practical, solid skills will you be considered for the next phase of the interview.

You are required to complete the assignment and post it in your GitHub repository. Please share the links with us for review by the specified date and time. Upon successful review, you will be invited to a virtual session to demonstrate your solution and answer follow-up questions.

You may use AI tools (code assistants, chat tools or similar). Be prepared to explain any part of your submission in the live session.

All API endpoints required for this assignment are documented in the Postman collection below. The collection provides details on request methods, required fields, and example payloads to be used when building your Android application.

- API Collection

Assignment

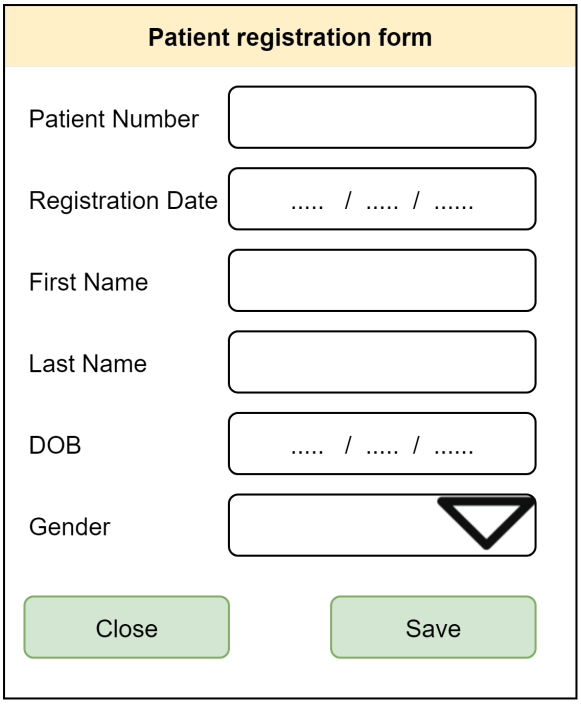
Marks will be awarded for clarity, comments, and ingenuity. You are NOT expected to submit a production-ready application. However, you are
encouraged to submit. Every part of the submission must be something you can explain and defend.

Develop an Android application to register patients, take vital signs (including height, weight, and BMI), collect additional information through two Assessment forms, and display a patient listing. The application will contain five(5) pages, namely:

- Patient registration page
- Vitals form
- Overweight assessment form
- General assessment form
- Patient Listing

Patient registration page

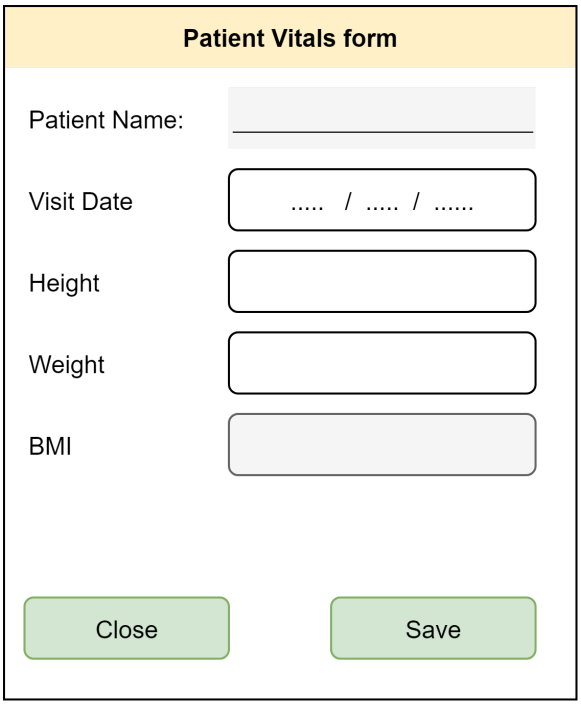
- Screen Mockup



- It contains a Registration form that captures the following fields:
    - Patient ID (This is a unique field and cannot be shared between patients)
    - Registration date
    - First name
    - Last name
    - Date of birth
    - Gender
- A patient can only be registered once (1)
- Upon saving the Patient Registration details, load the Vitals page(1)

Vitals page

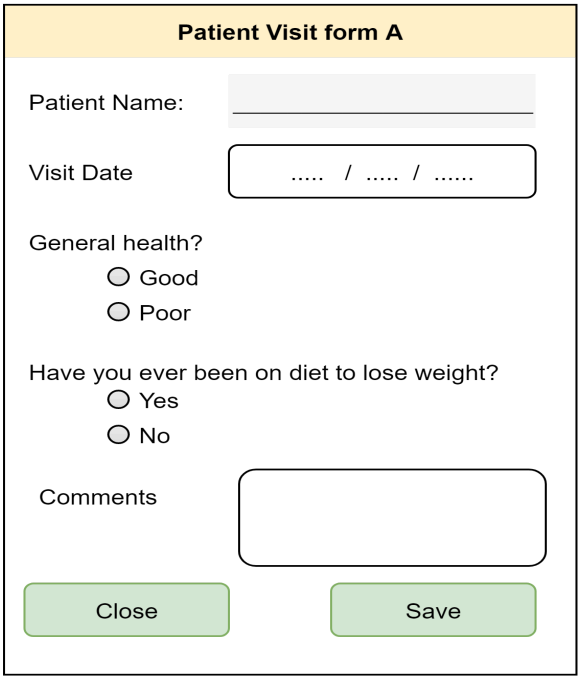
- Screen Mockup



- It contains a Patient Vitals form that captures the following fields:
    - Visit Date
    - Height (In CM)
    - Weight (In KG)
    - BMI (This value is auto-calculated based on Height & Weight: ie. BMI = Weight(kg)/ Height(M)^2)
- All fields are mandatory
- A single patient can have multiple submissions for the vitals form, but on different dates.
- Upon saving the vitals form, if the patient's BMI (BMI = kg/m2) is less than 25, the app should automatically load the General Assessment Page; otherwise, load the Overweight Assessment Page.

General Assessment Page

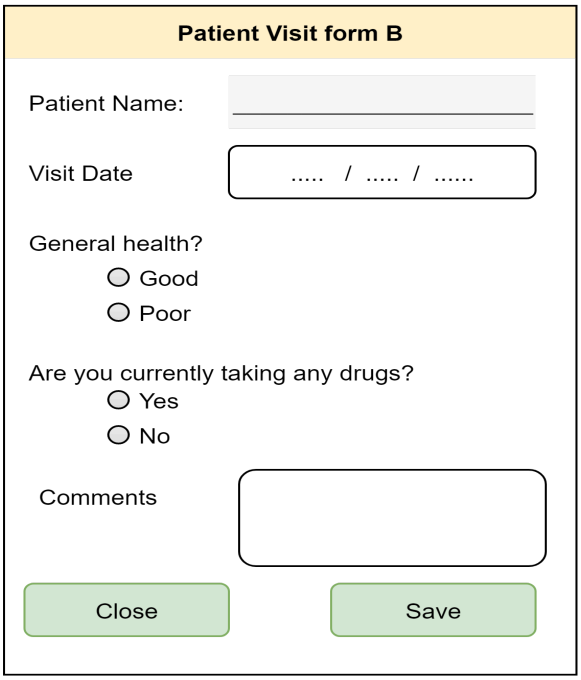
- Mockup



- Contains the General Assessment Form, which collects the following information:
    - Visit Date
    - General health (Good/Poor)
    - Have you ever been on a diet to lose weight? (Yes/No)
    - Comments
- A single patient can have multiple submissions for the General assessment form, but on different dates.
- The page is only visible if BMI < 25
- All fields are mandatory
- Upon saving this form, load the patient listing page

Overweight Assessment Page

- Screen Mockup



- Contains the Overweight assessment form, which collects the following information:
    - Visit Date
    - General Health (Good/Poor)
    - Are you currently using any drugs (Yes/No)
    - Comments
- A single patient can have multiple submissions for the Overweight assessment form, but on different dates.
- Should only be displayed for patients whose BMI is equal to or greater than 25
- Upon saving the form, load the patient listing page.

Patient Listing Page

- Screen Mockup

Patient Listing

Date ..... / ..... / .....

Patient Name	Age	BMI Status
John Doe	35	Normal
Jane Doe	18	Overweight
James Doe	20	Underweight
Julia Doe	28	Normal

- Should display a list of patients showing the following columns:
  - Patient Name
  - Age
  - Last BMI Status
- Let the BMI status be either “Underweight”,” Normal”, or “Overweight”. Underweight patients have a BMI of less than 18.5; normal patients have a BMI of 18.5 or greater and less than 25, and a BMI of 25 or greater for overweight patients.
- The report should have the ability to filter records by the visit date.

Form submission

Upon saving the forms, the application should:

- Store data locally (Room/SQLite database)
- Trigger data submission to a backend service defined in the provided Postman collection.

Deliverables

In addition to the application, your submission must include the following.

- Git history. Commit as you go. We expect a history spanning at least two working sessions; a single bulk commit will be queried.
- Screen recording. A recording of under 5 minutes (any format) of the app running on a device or emulator, covering the full flow from registration to the listing filter.

- Tests. Unit tests for BMI calculation and BMI-status classification including boundary values; unit tests for form validation (mandatory fields, duplicate Patient ID, one submission per date); one Room DAO test; one repository test using a faked or mocked API.
- Design note (DESIGN.md, one page maximum) covering: (1) the two or three design decisions you made and why (architecture, libraries, data model); (2) what you deliberately left out or simplified, and why. Keep it factual and short. Trade-offs and alternatives will be discussed with you in the live session.

Preferred Tech stacks

Kotlin. Kotlin Multiplatform is preferred; an Android-only Kotlin submission is accepted. Either way, state in your design note which parts of your code could move to a shared KMP module and which could not.

Accessing the Backend API Service

To interact with the backend service, use the provided Postman collection, which contains all the endpoints, request methods, payload examples, and sample responses required for this practical.

How to Access the API Documentation and Collection:

- Visit the postman documentation link
- Import the collection into your Postman workspace
- Review the available endpoints and payload formats
- Use the example requests to test connectivity and understand response structures before integrating them into your application.

API Details

- Base URL:
    - https://patientvisitapis.intellisoftkenya.com/api/
- Endpoints Overview
    - POST user/signup - Registers a new user
    - POST user/login - Authenticate and obtain an access token
    - POST patients/register - Registers a new patient
    - GET patients/list - Retrieve a list of registered patients
    - GET patients/show/{id} - Displays specific patient information
    - POST vitals/add - Records patient's vitals
    - POST visits/view - Display all the visits for a particular day
        - Requires a visit date
    - GET visits/add - Create a patient visit, either form A or B
- Usage Notes
    - Ensure all requests include the necessary authentication token (if applicable)
    - Required request body parameters and sample payloads are available in the Postman collection.
    - The API follows REST conventions and returns JSON responses
    - Use Postman's environment variables to manage the base URL and authentication token efficiently.

How will your work be judged? You will be judged 40% on the submission (correctness, validation, error handling, tests, code quality, and a properly designed, functional and attractive

UI) and 60% on the live session, where you will walk us through your code, extend it, review some of ours, and answer questions about how it would behave in real use.